
**title: “Monitoring simultâneo 4G / 5G SA / 5G NSA - Plan” type: feat
date: 2026-08-12 status: superseded superseded_by: docs/plans/2026-08-14-001-feat-5g-nrcell-nrducell-monitoring-plan.md**

SUPERADO em 14/08/2026 por `docs/plans/2026-08-14-001-feat-5g-nrcell-nrducell-monitoring-plan.md` .

A captura dos quatro HARs de `har-5g-oss/` mostrou que **não existem tasks “SA” e “NSA”** em nenhum dos dois OSS. A divisão real é por tipo de objeto — **NR Cell** e **NR DU Cell** — e os dois conjuntos de contadores são disjuntos e já corretamente cobertos pelo catálogo atual. Mantido como histórico: o diagnóstico do `ValueError` de duas tasks 5G e a hipótese de colisão de `objNo` estavam certos (a colisão foi confirmada: 63 `objNo` compartilhados entre as tasks 2241 e 2242 em OUTRAS). Erradas eram a divisão SA/NSA do catálogo e a remoção da subtração de volume SA.

Monitoring simultâneo 4G / 5G SA / 5G NSA - Plan

Contexto e decisões assumidas

Hoje o sistema só reconhece duas tecnologias (`4G` , `5G`) em toda a cadeia de coleta — `core/collector.py` normaliza qualquer célula/task para um desses dois valores, `core/kpi_formulas.py` só cataloga fórmulas para `4G` / `5G` , e a task PM Monitoring é única por tecnologia (`_configured_pm_tasks` lança `ValueError` se houver duas). Configurar as tasks SA e NSA hoje faz as duas caírem no rótulo `5G` e o ciclo de KPI inteiro (4G incluso) passa a falhar a cada 120s.

Este plano assume as decisões já confirmadas na análise anterior:

- **Cardinalidade célula↔task:** uma célula 5G física participa das duas tasks (SA e NSA), reportando sob `objNo` possivelmente diferente em cada uma. Isso é resolvido **dinamicamente** pela descoberta já existente (casamento por nome de objeto, ver `_resolve_monitoring_cell`) — **não** é necessário criar campos manuais de `obj_no` por tecnologia no cadastro do evento, porque, na prática, nenhum evento cadastrado hoje preenche `obj_no` manualmente (o importador de planilha do cadastro nunca gerou esse campo); a descoberta dinâmica é o caminho real de produção. O que precisa ser corrigido é a própria descoberta, que hoje tem duas falhas que impediriam SA/NSA de resolver:
 1. Ela compara a tecnologia da célula (`4G` / `5G` , genérica) com a tecnologia exata da task (`5G_SA` / `5G_NSA`) por igualdade estrita — nunca bateria, e toda célula 5G ficaria “não mapeada” nas duas novas tasks.
 2. O cache de `objNo` resolvido é global (`{objNo: célula}`), sem separar por task. Se duas tasks reaproveitarem o mesmo número pequeno de `objNo` para células fisicamente diferentes (comum em numeração local por task no OSS), a segunda task herdaria silenciosamente o mapeamento da primeira.
- **Separação visual:** um filtro/aba de tecnologia (4G / 5G) acima da lista de sites, filtrando por célula. A divisão SA vs. NSA fica a cargo do seletor de métrica (cada métrica já é escopada a uma tecnologia exata) — a aba de site não precisa descer a esse nível, porque o “site” é uma unidade de cobertura física, não de task.
- **Mapeamento de KPI por tecnologia (a validar com captura real do OSS, mesmo cuidado já registrado em `MEMORY.md` / `ERRORS.md` de não presumir contrato sem medir):** `accessibility` , `drop_rate` , `availability` , `utilization_dl` , `utilization_ul` , `user_count` , `interference_ul` → `5G_SA` ; `throughput_dl` , `throughput_ul` , `traffic_volume_dl` , `traffic_volume_ul` → `5G_NSA` . Isso também **elimina** a gambiarra atual de subtração (`traffic_volume_dl_sa = N.ThpVol.DL - N.NSA.ThpVol.DL`), já que SA e NSA passam a ser tasks nativas com contadores próprios, sem sobreposição.

Divisão em fases

O trabalho tem três superfícies bem separadas — correção do coletor (código sensível, já documentado com vários incidentes de coleta silenciosa), formulário de cadastro de evento (outro arquivo, outro processo) e o dashboard do operador (visual) — cada uma testável e entregável de forma independente. Por isso, divido em 3 fases sequenciais: a Fase 1 é o alicerce de corretude e pode ser validada sozinha (testes automatizados + evento de teste carregado manualmente); a Fase 2 destrava o cadastro em produção sem depender da Fase 3; a Fase 3 só faz sentido depois que o backend já entrega `5G_SA` / `5G_NSA` de verdade.

Fase 1 — Coleta: modelo de 3 tecnologias no coletor

Explicação simples

Ensinar o coletor a tratar `4G` , `5G_SA` e `5G_NSA` como três tecnologias distintas de verdade — cada uma com sua própria task PM, seu próprio catálogo de fórmulas de KPI e sua própria resolução de célula —

em vez do enum binário atual que faz a segunda task "5G" derrubar o ciclo inteiro.

Escopo detalhado

Código

- `core/collector.py`
 - Nova função `_normalize_task_technology(value)` (separada da existente `_normalize_cell_technology`): reconhece `4G / LTE` → `"4G"`; `5G_SA / 5G-SA / SA` → `"5G_SA"`; `5G_NSA / 5G-NSA / NSA` → `"5G_NSA"`. Usada exclusivamente para normalizar `integration.pm_tasks[].tech`, vindo da configuração do evento — não tenta inferir a partir de nome de célula (célula não carrega essa informação).
 - Novo mapa `_TECH_FAMILY = {"4G": "4G", "5G_SA": "5G", "5G_NSA": "5G"}`.
 - `_configured_pm_tasks`: passa a chamar `_normalize_task_technology` (em vez de `_normalize_cell_technology`) ao ler `pm_tasks[].tech`. A trava de “uma task por tecnologia” continua igual, agora operando sobre os 3 valores possíveis — ela já é o comportamento certo (uma SA, uma NSA, uma 4G), só precisa dos 3 rótulos.
 - `_resolve_monitoring_cell(obj_no, obj_name, technology)`:
 - Cache `self._obj_to_cell` passa a ser chaveado por `(technology, obj_no)` em vez de `obj_no` isolado, eliminando a colisão entre tasks que reaproveitam a mesma numeração.
 - O filtro de candidatos por nome passa a comparar **família** (`_TECH_FAMILY[technology]` contra `_TECH_FAMILY[metadata["technology"]]`), não a tecnologia exata — assim uma célula cadastrada como `5G` responde tanto pela task SA quanto pela NSA.
 - `_request_objects_for_task`: ajustar a leitura de `_obj_to_cell.items()` para a nova chave em tupla, mantendo o filtro por tecnologia exata da task.
 - `__init__` / mapeamento estático de `_cell_metadata`: sem mudança de estrutura — continua guardando a família (`4G / 5G`) por célula, exatamente como hoje.
 - `MockCollector.collect_kpis`: passa a marcar `technology` em cada linha gerada (hoje não marca nenhuma) e, para células da família `5G`, gera duas rodadas de métricas — uma sob `5G_SA` (accessibility/drop/availability) e outra sob `5G_NSA` (throughput/volume) — replicando o comportamento real de uma célula reportando nas duas tasks. Necessário para validar a Fase 3 depois, em modo `--mock`, sem VPN.
- `core/kpi_formulas.py`
 - Substituir as 15 entradas atuais de `CATALOG` com `technology="5G"` por duas famílias: `5G_SA` (`accessibility`, `drop_rate`, `availability`, `utilization_dl`, `utilization_ul`, `user_count`, `interference_ul`) e `5G_NSA` (`throughput_dl`, `throughput_ul`, `traffic_volume_dl`, `traffic_volume_ul`).
 - Remover `_n_volume_dl_sa / _n_volume_ul_sa` e as definições `traffic_volume_dl_sa / _nsa / ul_sa / _nsa` (a subtração deixa de ser necessária: `traffic_volume_dl / _ul` em `5G_NSA` usam `N.NSA.ThpVol.DL / .UL` diretamente, via `_single(...)`, do mesmo jeito que `throughput_dl / _ul` já usam `_n_thp_dl / _n_thp_ul`).
 - `production_ready=False` deve ser reavaliado métrica a métrica ao confirmar os contadores reais da task NSA (hoje só `throughput_dl / _ul` estão marcados assim; manter marcação pendente para o que não for validado).

Interface

- Nenhuma nesta fase — mudança de backend, validada por evento carregado manualmente (JSON) e pelos testes.

Testes

- `tests/test_kpi_monitoring.py` : estender o fixture `_event()` para 3 tasks (`4G` , `5G_SA` , `5G_NSA`) com uma célula 5G que aparece nas duas tasks 5G sob `objNo` diferentes.
- Novo teste: duas tasks (`5G_SA` `task_id=20`, `5G_NSA` `task_id=30`) reaproveitando o **mesmo** `objNo` para células **diferentes** — provar que a linha da task 30 não herda a célula resolvida pela task 20 (prova direta da correção do cache por (`technology`, `obj_no`)). a) Antes da correção, este teste deve falhar — igual à disciplina já registrada em `ERRORS.md` ("reverter a correção e confirmar que o teste falha").
- Novo teste: célula cadastrada como `5G` resolve corretamente contra uma task `5G_SA` e contra uma task `5G_NSA` (prova da comparação por família).
- Novo teste: `_configured_pm_tasks` aceita as 3 tasks simultaneamente e ainda rejeita duas tasks para a mesma tecnologia exata (ex.: duas `5G_SA`).
- `tests/test_collector.py` (se aplicável) e cobertura de `kpi_formulas.catalog_for_api()` garantindo que `5G_SA` / `5G_NSA` aparecem e `5G` não aparece mais.

Como validar e testar

Automatizado

```
.venv\Scripts\python.exe -m pytest tests/test_kpi_monitoring.py tests/test_kpi_formulas.py tests/test_collector.py -v  
.venv\Scripts\python.exe -m pytest tests/ -v
```

A suíte completa não deve ganhar falhas novas além das duas já documentadas em `MEMORY.md` (`TestMockCollectAlarms` , `test_resolve_base_url_pelo_catalogo`).

Visual

1. Rodar `python main.py --mock` (ou `--mock --dev`) com um evento de teste que tenha ao menos um site com célula `5G` .
2. Confirmar no log/ `get_status` (indicador de sincronização do app) que o ciclo de KPI reporta `state: "data"` sem erro — hoje, com uma 2ª task "5G", ele entraria em erro a cada ciclo.
3. Conferir no seletor de métricas (mesmo antes da Fase 3 dividir os grupos visualmente) que aparecem métricas com `5G_SA` e `5G_NSA` no `title` /`tooltip` (`Api.getKpiCatalog`), confirmando que o catálogo novo está sendo servido.

Sugestão de commit

```
feat: separate 5G SA and NSA as distinct monitoring technologies in the collector
```

Fase 2 — Cadastro do evento: campos das 3 tasks PM

Explicação simples

Trocar o campo único “PM Task” do formulário de cadastro/edição de evento (`server_frontend/index.html`) por três campos — 4G, 5G SA e 5G NSA — para que o operador consiga configurar as 3 tasks sem editar o JSON à mão.

Escopo detalhado

Código / Interface (`server_frontend/index.html` — é o único arquivo do formulário, HTML+JS inline)

- Substituir o input único `#pm-task-id` por três inputs numéricos opcionais, com rótulo por tecnologia: `#pm-task-4g`, `#pm-task-5g-sa`, `#pm-task-5g-nsa`. Mantém o padrão visual dos demais campos do formulário (mesmos containers/labels usados por `oss-cliente` / `oss-region` etc.).
- No envio do formulário (`btnSubmit` handler): montar `integration.pm_tasks = [{tech, task_id}, ...]` filtrando os campos vazios, em vez do atual `integration: { pm_task_id }`. Task inválida/não numérica não entra na lista (mesma tolerância que `_configured_pm_tasks` já tem no backend para tasks ausentes).
- Em `editEvent(id)`: popular os 3 campos a partir de `event.integration.pm_tasks` (procurando por `tech`); se o evento ainda for legado e só tiver `integration.pm_task_id`, popular **somente** o campo 4G a partir dele (o comportamento histórico de hoje), sem reescrever nada até o operador salvar de novo.
- `renderEvents`: trocar a linha `PM Task: X` do card do evento por um resumo com as 3 tasks presentes, ex. `4G #10 · 5G SA #20 · 5G NSA #30` (omitindo as ausentes).
- Fora de escopo nesta fase (ver “Contexto e decisões assumidas”): nenhum campo novo por célula/planiha — `obj_no` continua não sendo inserido manualmente; a Fase 1 já cobre a resolução dinâmica por task.
- `server.py`: sem mudança — o endpoint `/api/events` já grava o JSON recebido sem validar o formato de `integration`.

Testes

- Não há suíte Python cobrindo `server_frontend/index.html` hoje (é servido por `server.py`, sem testes automatizados dedicados). Escopo desta fase é validação manual/visual (abaixo). Se o projeto já usa Playwright para telas do app principal (`tests/test_frontend_collection_ui.py`), avaliar reaproveitar o mesmo padrão de servidor temporário + Chromium headless para um teste mínimo de `server_frontend` cobrindo: preencher as 3 tasks, salvar, recarregar a lista e confirmar que os 3 valores persistiram no card do evento. Esse teste é desejável, não bloqueante, dado que hoje não existe nenhuma cobertura de referência para esta tela.

Como validar e testar

Visual

1. Rodar `python server.py` (ou o comando equivalente já usado pela equipe para subir o servidor central) e abrir `http://localhost:8000`.
2. Cadastrar um evento novo preenchendo os 3 campos de task (4G, 5G SA, 5G NSA) e salvar; confirmar no card da lista que as 3 tasks aparecem.
3. Editar esse mesmo evento e confirmar que os 3 campos vêm pré-preenchidos corretamente.
4. Abrir um evento legado já existente (criado antes desta mudança, com `pm_task_id` simples) e confirmar que o campo 4G aparece populado com o valor antigo, e os campos SA/NSA aparecem vazios.
5. Salvar esse evento legado e conferir no JSON gravado (`server_data/events/<id>.json`) que ele passou a usar `integration.pm_tasks`.

Automatizado

- Se o teste Playwright mínimo descrito acima for implementado:

```
.venv\Scripts\python.exe -m pytest tests/test_server_frontend_event_form_ui.py -v
```

- Caso contrário, apenas confirmar que a suíte completa (`pytest tests/ -v`) continua sem regressões, já que esta fase não toca em código Python de backend.

Sugestão de commit

```
feat: add 4G/5G SA/5G NSA task fields to event registration form
```

Fase 3 — Dashboard do operador: separação visual 4G/5G e métricas SA/NSA

Explicação simples

No app principal (o que o operador olha durante o evento), dividir de fato o seletor de métricas em três grupos reais (hoje “5G SA / NSA” é um único grupo rotulado, mas mistura tudo) e adicionar uma aba/filtro 4G × 5G acima da lista de sites, para não misturar visualmente as duas tecnologias.

Escopo detalhado

Código

- `frontend/js/kpi.js`
 - `_loadKpiCatalog` : trocar o mapa de grupos de `[["common",...], ["4G",...], ["5G",...]]` para `[["common",...], ["4G",...], ["5G_SA",...], ["5G_NSA",...]]`, com rótulos "Adicionais 4G", "5G SA", "5G NSA".
 - Nova constante `TECH_FAMILY = { "4G": "4G", "5G_SA": "5G", "5G_NSA": "5G" }` (espelhando a do backend) e um helper `_cellTechFamily(cell)` portado da mesma lógica de inferência por nome já usada em `server_frontend/index.html` (`resolveTechFreqFromId`), para não reinventar a heurística em dois lugares com regras diferentes.
 - `_renderSiteList` : aplicar o filtro de tecnologia ativo (`State.techFilter`) — um site permanece visível se tiver ao menos uma célula da família selecionada (ou sempre, se o filtro for "Todas"); célula sem família identificável é tratada como visível em qualquer filtro (falha aberta, não esconde dado por engano).
 - `_populateCellSelector` : mesmo filtro aplicado à lista de células do site selecionado.
- `frontend/js/state.js` : novo campo de estado `techFilter` (`"all" | "4G" | "5G"`, default `"all"`), com o evento `change:techFilter` já coberto pelo mecanismo genérico de `State.on / State.set` existente.
- `frontend/index.html` : novo bloco de abas acima de `#site-list`, reaproveitando o mesmo padrão markup/classe de `#time-tabs / .time-tab` já usado para as janelas temporais (ex.: `#tech-tabs` com botões "Todas", "4G", "5G").
- `frontend/css/main.css` : estilo do `.tech-tab` (reutilizando as regras existentes de `.time-tab / .active`) e duas cores de badge para 4G/5G, no mesmo espírito de `STATUS_COLORS / site-dot` já usado na lista de sites — usado tanto na aba quanto em um pequeno indicador por célula no `cell-selector`.

Interface

- Aba "Todas / 4G / 5G" fixa acima da lista de sites, com o mesmo padrão visual das abas de janela temporal já existentes (ativa/inativa).
- Seletor de métricas com 3 grupos reais no dropdown (4G / 5G SA / 5G NSA), cada opção já mostrando a tecnologia no texto (`_getMetricSuffix / title` já fazem isso, sem mudança adicional necessária aí).
- Lista de sites e seletor de célula respeitando a aba ativa.

Testes

- `tests/test_frontend_collection_ui.py` (ou um novo arquivo dedicado, seguindo o mesmo padrão de servidor temporário + Chromium headless):
 - o seletor de métricas expõe `optgroup` para `5G SA` e `5G NSA` separadamente (e não mais um único "5G SA / NSA");
 - clicar na aba "4G" esconde da lista sites/células sem nenhuma célula 4G e mantém os demais;
 - clicar na aba "5G" tem o comportamento espelhado;
 - voltar para "Todas" restaura a lista completa.
- Reaproveitar/estender os cenários mockados de `frontend/js/bridge.js` (mesmo padrão já usado pela Fase de VIP) para garantir sites com composição mista de células 4G/5G determinística.

Como validar e testar

Automatizado

```
.venv\Scripts\python.exe -m pytest tests/test_frontend_collection_ui.py -v  
.venv\Scripts\python.exe -m pytest tests/ -v
```

Visual

1. Rodar `python main.py --mock --dev` com um evento que tenha sites com células 4G e 5G (a MockCollector da Fase 1 já gera medições `5G_SA` / `5G_NSA` para as células 5G).
2. Abrir o seletor de métricas e confirmar 3 grupos reais: "Métricas comuns", "Adicionais 4G", "5G SA", "5G NSA".
3. Clicar na aba "4G": confirmar que só sites/células 4G aparecem na lista lateral e no seletor de célula do gráfico; clicar em "5G" e confirmar o inverso; voltar para "Todas".
4. Selecionar uma métrica de `5G_SA` (ex. acessibilidade) e depois uma de `5G_NSA` (ex. throughput) e confirmar que o gráfico atualiza com dados distintos para a mesma célula.
5. Conferir visualmente que as abas 4G/5G têm cores distintas e consistentes com o restante da paleta de status já usada na lista de sites (sem introduzir uma paleta nova desalinhada).

Sugestão de commit

```
feat: split 4G/5G view and SA/NSA metrics in the operator dashboard
```

Riscos e pontos em aberto

- **Mapeamento exato de contador** → **SA/NSA** (Fase 1) foi proposto por dedução a partir do pedido ("SA para drop/ acessibilidade, NSA para thpt/volume"), não confirmado contra uma captura real da tela de Monitoring do iManager para as duas tasks. Recomenda-se validar isso ao vivo antes de fechar a Fase 1, do mesmo jeito que o contrato do VIP foi validado por HAR (ver `MEMORY.md`).
- `utilization_dl` / `utilization_ul` / `availability` foram atribuídos a `5G_SA` por suposição (contadores de PRB/disponibilidade de célula tendem a vir do lado de acesso), mas não foram citados explicitamente pelo usuário — vale confirmar se a task NSA também os expõe e se fazem mais sentido lá.
- **Nomenclatura exata da tecnologia** (`5G_SA` vs. `5G-SA` vs. `SA`) é uma escolha de implementação deste plano; qualquer valor funciona, desde que único e usado de forma consistente entre `core/kpi_formulas.py`, `core/collector.py`, `server_frontend/index.html` e `frontend/js/kpi.js`.